Lab2 实验报告

本实验主要解决三个问题：内存管理、进程切换、进程生命周期与调度

1 整体调用逻辑

可以分成两条线来描述：

1. **启动与创建流程**：start_kernel -> task_init (创建 pid 0) -> kernel_thread (创建 kthreadd, pid 1) -> test_sched (创建更多测试进程)。
2. **调度与抢占流程**：一旦时钟中断开启，任何正在运行的进程都可能被时钟中断打断 -> 进入 _traps -> trap_handler -> schedule() -> switch_to() 切换到新进程。

1.1 启动流程

这个阶段从 head.S 开始，到 start_kernel 的主要工作完成为止。全程只有一个执行流。

1.1.1 head.S

打开栈，设置中断入口，初始化核心服务 (call mm_init, call task_init，把内存管理器和第一个进程（0 号进程）的数据结构准备好)，最后调用start_kernel

此时有了一个代表 start_kernel 自身的 0 号进程。但整个系统仍然是单线程的。

1.1.2 start_kernal

完成系统初始化，并“创造”出其他进程，为多任务做准备。

调用 kernel_thread 创建了 kthreadd (pid 1) 和 test_sched (pid 2) 等新进程。此刻，task_list 里已经有了多个待运行的进程。

1.2 调度循环

- **目标**：系统进入一个由时钟中断驱动的、永不停歇的调度循环中。
- **关键动作 (这是一个循环)**:**时钟中断发生**: 无论当前哪个进程在运行，都会被强制暂停。**内核夺权 (trap_handler)**: 控制权回到内核。**大脑决策 (schedule)**: 内核的调度器根据时间片轮转算法，选出下一个应该运行的进程。**灵魂互换 (switch_to)**: 内核通过汇编代码，完成两个进程上下文的切换。**新进程运行**: 新的进程从它上次暂停的地方继续执行，直到下一次时钟中断的到来...

此阶段的特点：系统进入了一个稳定的、周而复始的状态。start_kernel 这个函数可能再也不会被调度到了（因为它最后会调用 do_exit 或 cpu_idle）。CPU 的控制权在不同的内核线程之间不断地传递，宏观上实现了多任务。

head.S中，开启内存调用，并调用栈

之后调用mm_init：初始化Buddy System

接着调用test_list：检测Task1

再调用task_init：

- 使用 `kmem_cache_create()` 创建 `task_struct` 对象缓存池，并分配一个新的 `task_struct` 结构体作为初始进程。
- 初始化该进程的各项字段，包括 `pid`、`stack`、`state`、`se`（调度实体，见 Part4）等。
- 将该进程设置为当前运行进程，并将其加入进程链表（`task_list`）。
- 完成后，`start_kernel` 将作为第一个 Task，进而能够通过它切换到其他进程，为后续进程管理和调度打下基础。

之后回到head.S

最后调用start_kernal，这是第一个进程，pid=0

> 运行后，kernal_thread调用了kthreadd：第二个进程，设置新进程的上下文
>
> kernal_create: test_sched：

do exit, 进入schedule，进行时间片轮转。
首先，计算当前时间片剩余时间，以及总共运行时间。然后开始调度。

2 内存管理

2.1 前置知识

内存管理上，我们要避免内存碎片化

所以采用两层架构：buddy system管理较大的内存请求（批发商），object Cache管理较小的内存请求（零售商）。

buddy system以2的幂次发页表，如果页表周围有一样大小的，就合并

Object Cache是小块串起来，比较灵活，需要时把头取下来即可

2.2 Task 1：实现双向链表

2.2.1 观察 `LIST_HEAD` 宏，你会发现这应该是一个**带哨兵**的双向链表。思考如果不带哨兵，它应当该如何实现？

其实总体差不多，结构定义也不变，只是比如在插入过程中要加入特判，初始化时没有节点。属于细节区别

2.2.2 实现细节

此处较为简单，所以省略描述

1. 该处为添加

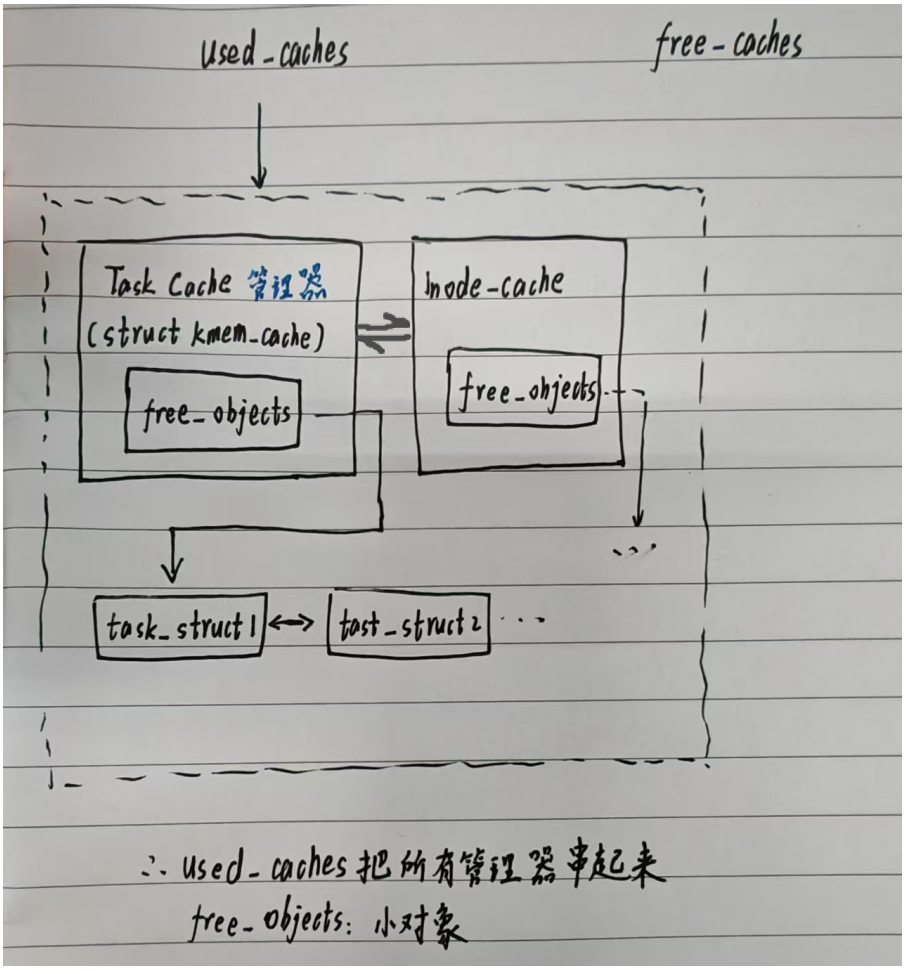
```
static inline void list_add(struct list_head *new, struct list_head *head)
{
  /* Lab2 Task1 */
  new->next = head->next;
  head->next->prev = new;
  head->next = new;
  new->prev = head;
}
```

2. 此处要特别注意，对于遍历起点的确定，以及终点的判断，不是从head开始

```
/**
 * @brief 遍历链表
 * @param pos 循环变量
 * @param head 链表头
 */
#define list_for_each(pos, head) \
  for (pos = (head)->next; pos != (head); pos = pos->next) /* Lab2 Task1  */
//当然，这个地方用循环是比较愚蠢的方法，因为我之前没完全理解这个双向链表是长什么样的，其实完全不用循环的
```

2.3 动手做：理解空闲链表

如下图所示，全局的 used_caches 链表串联起了所有的缓存管理器（如 task_cache）。每个管理器内部，又通过 free_objects 链表，管理着从物理页上切割好的空闲对象。



3 进程切换与 Trap 处理

3.1 背景知识

1. 存什么？schedule存哪些参数

   ra（返回地址），sp（栈地址，因为每一个进程栈独立），s1-s12。schedule类似于交接班

2. 内核抢占的时机：中断时，何时调用？

   在处理完中断，恢复之前（此时该存的都存了）

3. 切换的场景

内核会以Trap的名义，强行切换进程。

4. 并发保护
   不能破坏原子操作。schedule本身也是原子的
5. 任务的数据结构（Task struct：pid、thread (我们之前讨论的上下文)、stack 指针等等）
6. 快速访问（使用TP寄存器）

3.2 Task 2：设置初始进程

task_init主要做的是初始化，是系统启动后的第一个进程

```
void task_init(void)
{
  task_cache =
    kmem_cache_create("task_struct", sizeof(struct task_struct));

  /* Lab2 Task2 */
  // 分配一个新的 task_struct 结构体作为初始进程
  struct task_struct *init_task = kmem_cache_alloc(task_cache);

  // 初始化进程的各项字段
  init_task->pid = 0;  // 初始进程 PID 为 0
  init_task->state = TASK_RUNNING;  // 设置为运行状态

  // 分配内核栈 - 使用一页内存作为栈
  init_task->stack = NULL;

  // 初始化调度实体
  init_task->se.runtime = 0;
  init_task->se.sum_exec_runtime = 0;

  init_task->thread.ra = (uint64_t)__kthread;

  // 初始化进程链表节点
  INIT_LIST_HEAD(&init_task->list);

  LOG_ERR("list pos %p\n", &init_task->list);

  // 将该进程设置为当前运行进程
  current = init_task;

  uint64_t flags;
  interrupt_save(&flags);

  // 将其加入进程链表
  list_add_tail(&init_task->list, &task_list);

  interrupt_restore(&flags);

  // 设置 idle_task 指针
  idle_task = init_task;
}
```

另外，由于trap程序也被打断，所以需要额外保存两个寄存器，sstatus sepc。

这是因为 trap_handler 现在可能会调用 schedule 切换走。如果当前中断上下文（比如 sstatus 里的 SIE 位）不被保存，那么当任务很久以后被切换回来，sret 恢复的就是一个错误的、属于其他任务的 sstatus，这会导致中断状态混乱，系统崩溃。

故在entry.S中增大栈空间，改为

```
addi sp, sp, -272
```

4 进程生命周期

4.1 Task 3：实现进程复制与加载

首先理解目的，该task是为了创建进程，而unix分为两部，即fork()和exec()，分别对应复制当前进程，以及在当前进程的上下文中加载并执行一个新的程序。

4.1.1 第一步，是实现copy_process

其功能是实现进程复制。所以需要先分配相应的进程结构体、内核栈、进程号。

```
struct task_struct *copy_process(struct task_struct *old)
{
  // 分配一个新的 task_struct 结构体
  struct task_struct *new_task = kmem_cache_alloc(task_cache);
  if (!new_task) {
    return NULL;
  }

  // 分配新的内核栈
  new_task->stack = alloc_page();
  if (!new_task->stack) {
    kmem_cache_free(task_cache, new_task);
    return NULL;
  }
```

创建完以后，进行初始化，设置进程状态为运行态，并添加到全局进程链表中

```
  // 分配唯一的进程号
  new_task->pid = create_pid();

  // 初始化调度相关字段
  new_task->se.runtime = 0;
  new_task->se.sum_exec_runtime = 0;

  // 初始化进程状态
  new_task->state = TASK_RUNNING;

  // 初始化进程链表节点
  INIT_LIST_HEAD(&new_task->list);
```

```
  uint64_t flags;
  interrupt_save(&flags);

  // 将新进程加入进程链表
  list_add_tail(&new_task->list, &task_list);

  interrupt_restore(&flags);

  return new_task;
}
```

至此，即创建一个与原进程具有相同属性的新进程。

4.1.2 第二步，实现kernel_thread

用于创建一个内核线程。要求调用 `copy_process()` 创建新进程，并根据对 `__kthread()` 的理解设置新进程的初始上下文。

通过精心设置新任务的 thread.ra 指向蹦床函数 kthread，并把真正的目标函数和参数存入 s0 和 s1，我们实现了一个巧妙的接力：switch_to 切换到新任务后，会 ret 到 kthread，kthread 再把参数从 s 寄存器搬到 a 寄存器，最后通过 ret (或者 call) 跳转到真正的目标函数。这个过程完美地初始化了新线程的执行环境

首先，我们来看__kthread()，它用于从内核线程初始上下文跳转到内核线程函数

```
//启用中断后，将保存在s0和s1中的返回地址和参数恢复，然后返回到指定地址继续执行。
__kthread:
    csrs sstatus, SSTATUS_SIE
    mv ra, s0
    mv a0, s1
    ret //执行返回指令，跳转到ra指向的地址
```

所以说，我们需要将ra指向kthread 函数，并通过 s0 和 s1 寄存器传递参数给 kthread

```
struct task_struct *kernel_thread(int (*fn)(void *), void *arg)
{
  struct task_struct *new_task = copy_process(current);
  if (!new_task) {
    return NULL;
  }

  // 设置新进程的初始上下文，使其能正确跳转到 __kthread 并传参
  // 根据 Part2 的讨论，我们需要设置 ra 指向 __kthread 函数
  extern void __kthread(void);
  new_task->thread.ra = (uint64_t)__kthread;

  // 设置栈指针指向新分配的内核栈的顶部
  // RISC-V 栈是向下增长的，所以指向栈的最高地址
  new_task->thread.sp = (uint64_t)new_task->stack + PGSIZE;

  // 通过 s0 和 s1 寄存器传递参数给 __kthread
  // s1 保存函数参数 arg
  // s0 保存要执行的函数指针 fn
  new_task->thread.s[0] = (uint64_t)fn;  // s0 保存函数指针
```

```
  new_task->thread.s[1] = (uint64_t)arg; // s1 保存参数

  // 其余的 callee-saved 寄存器初始化为 0
  for (int i = 2; i < 12; i++) {
    new_task->thread.s[i] = 0;
  }

  return new_task;
}
```

5 调度器

此处我们实现的是RR 调度算法，即时间片轮转调度算法。

5.1 Task 4：实现时间片轮转调度和抢占

5.1.1 补全 `schedule()` 函数

首先，根据时间片轮转调度算法，每个进程被调度时，都要拥有时间片。调度时，更新时间片，并将当前任务移动到任务列表末尾。

```
static uint64_t last_sched_time = 0;
uint64_t current_time = clock();

//计算运转时间
uint64_t elapsed = current_time - last_sched_time;
last_sched_time = current_time; //更新上次调度时间

//计算总执行时间
current->se.sum_exec_runtime += elapsed;

//更新当前任务的剩余运行时间
if (elapsed < current->se.runtime) {
  current->se.runtime -= elapsed;
} else {
  current->se.runtime = 0;//结束了
}

/* 将当前任务移动到任务列表末尾，实现轮转调度 */
list_del(&current->list); //去头
list_add_tail(&current->list, &task_list); //丢到尾部

  /* Lab2 Task4 */
  // 从当前任务的下一个节点开始寻找
  struct task_struct *task;
  struct list_head *pos, *n;

  // 遍历整个进程链表一圈来寻找下一个可运行的进程
  if (current->se.runtime == 0) {
    list_for_each_safe(pos, n, &task_list){
      task = list_entry(pos, struct task_struct, list);
```

```
      // 如果找到了一个处于运行状态的进程，就选择它并跳出循环
      if (task->state == TASK_RUNNING && task != current) {
        next_task = task;
        break;
      }
      else if (task->state == TASK_DEAD && task != current) {
        // 如果找到了一个已经退出的进程，就释放它并跳过它
        release_task(task);
      }
    }
  }

  // 如果遍历完一圈后没有找到其他可运行的进程，
  // 并且当前进程仍然是可运行的，那就继续运行当前进程
  if (!next_task && current->state == TASK_RUNNING) {
    next_task = current;
  }



  /* 检查是否存在下一个任务 */
  if (next_task) {
    /* 确保下一个任务不是当前正在运行的任务 */
    if (next_task != current) {
      LOG_DEBUG("Schedule to [PID = %" PRIu64 "]\n",
          next_task->pid);
      /* 为下一个任务分配时间片 */
      next_task->se.runtime = TIME_SLICE;
      /* 执行任务切换 */
      switch_to(next_task);
    }
  } else {
    /* 没有找到合适的任务，记录错误并进入无限循环 */
    LOG_ERR("No suitable task found.\n");
    for (;;)
      ;
  }
  /* 恢复中断状态 */
  interrupt_restore(&flags);
}
```

5.1.2 `do_exit()` 和 `release_task()` 函数，实现内核线程的退出与销毁。

进程的销毁被分为 do_exit 和 release_task 两步。

前者在即将死亡的进程上下文中运行，它将自己标记为 TASK_DEAD 后就让出 CPU。

后者在另一个进程（通常是调度器或者专门的回收进程）的上下文中运行。

这种分离是必须的，因为一个进程无法安全地释放自己正在使用的内核栈。一旦栈被释放，do_exit 函数本身就无法继续执行了。

```c
noreturn void do_exit(int exit_code)
{
  /* Lab2 Task4 */
  // 将当前进程的状态标记为“死亡”
  current->state = TASK_DEAD;
  current->se.runtime = 0;
  // 调用调度器，切换到另一个进程
  schedule();

  (void)exit_code;

  // __builtin_unreachable() 告诉编译器这部分代码不会被执行，
  // 因为 schedule() 不会返回到这个已退出的进程中。
  __builtin_unreachable();
}
```

release_task函数，需要在中断保护下进行，以保证原子性。需要实现以下三点：

1. 从全局进程链表中删除指定任务
2. 释放该任务占用的内核栈内存页
3. 释放任务结构体本身占用的内存

```c
interrupt_save(&flags);

/* Lab2 Task4 */
// 1. 从全局进程链表中移除该任务
list_del(&task->list);

// 2. 释放该任务的内核栈所占用的内存页
if(task->stack)
  free_pages(task->stack);

// 3. 从 task_struct 的缓存中释放该任务的结构体
kmem_cache_free(task_cache, task);

interrupt_restore(&flags);
```

5.1.3 在 `trap_handler()` 的末尾调用 `schedule()`，从而启用内核抢占

```c
// 在处理完陷阱（特别是时钟中断）后调用调度器，以实现内核抢占
schedule();
```

5.2 动手做：计算响应比

```
[Lab2 Task5] Priority-based Scheduling Test

Process 1 started at 0, burst 10
11111111111111
Process 2 started at 1, burst 1
222211112222211111
Process 2 finished at 3
111111111
Process 3 started at 4, burst 2
33331111133333
Process 4 started at 6, burst 1
444411111333344444
Process 4 finished at 8
111133333
Process 3 finished at 9

Process 5 started at 9, burst 5
555511111555551111155551111555551111155555111115555511111555551111155555111115555111155555
Process 5 finished at 18
1111
Process 1 finished at 19
```

进程号 (逻辑)	进程 PID (实际)	到达时间 (Arrival)	实际运行时间 (Burst)	完成时间 (Completion)	周转时间	等待时间	响应比
1	3	0	10	19	19	9	1.9
2	4	1	1	3	2	1	2
3	5	4	2	9	5	3	2.5
4	6	6	1	8	2	1	2
5	7	9	5	18	9	4	1.8

平均周转时间 = （19+2+5+2+9）/5=7.4

平均等待时间=(9+1+3+1+4)/5 =3.6

平均响应比=(1.9+2+2.5+2+1.8)/5 =2.04

5.3 动手做：探究时间片对调度结果的影响

5.3.1 请你解释为什么 IPC 变低会影响调度结果

首先，存在两种时间。

1. **物理时间 (Wall-Clock Time)**：由 time 寄存器表示。QEMU 保证它以恒定的 10MHz 速率增长，不受模拟 CPU 运行速度的影响。
2. **执行时间 (Execution Time)**：指 CPU 执行指令、推进程序逻辑所花费的时间。它的快慢取决于 **IPC (Instructions Per Clock cycle)**。没有 GDB 时 IPC 高，CPU 跑得飞快；有 GDB 时 IPC 低，CPU 慢得像蜗牛。

时钟中断的触发和进程的创建请求都依赖物理时间。

IPC决定了CPU执行速度的快慢。当IPC很小时，CPU 慢悠悠地执行。test_sched 可能还在第一个 while 循环里打转，等待 clock() 走到那个时刻。结果，时钟中断的闹钟先响了。调度器被唤醒，发现还是只有老的进程，test_sched 还没来得及把新的创建请求提交上来。进而影响调度结果。

但上面这种情况比较夸张，一般情况下，开启/不开启调试时的调度结果相近。

5.3.2 请你思考可能发生的各种情况，然后尝试调整这两个宏的值，观察日志验证你的猜想。

5.3.2.1 将时间片调为10ms。此时，进程切换变得更加频繁。其实此举有负面影响，因为进程切换也是有开销的。

```
[Lab2 Task5] Priority-based Scheduling Test

Process 1 started at 0, burst 10
111111111
Process 2 started at 1, burst 1
121212121212121212
Process 2 finished at 3
111111111
Process 3 started at 4, burst 2
131313131
Process 4 started at 5, burst 1
3143143143143143143143143143143143
Process 5 started at 8, burst 5
3
Process 4 finished at 8
1513513513513
Process 3 finished at 9
5151515151511555151515151511515511515155151515151515151151515151515155151155115151515151
Process 5 finished at 18
11111
Process 1 finished at 19
```

5.3.2.2　此处，我们延长时间片的时间至2.5s，可以看到此时进程切换速度变慢，各进程的周转时间、等待时间及响应比都发生变化。

```
[Lab2 Task5] Priority-based Scheduling Test

Process 1 started at 0, burst 10
1111111111111111111111111111111111111111111111111
Process 2 started at 5, burst 1
222222222
Process 2 finished at 6
1111111111111111111111111
Process 3 started at 8, burst 2
333333333333333333
Process 3 finished at 10

Process 4 started at 10, burst 1
444444444
Process 4 finished at 11
1111111111111111111111111
Process 1 finished at 14

Process 5 started at 14, burst 5
5555555555555555555555555555555555555555555555555
Process 5 finished at 19
```